

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Student Name: M.Purna Chandra

Roll No.: A24126552095

Date: 16-08-2026

1. TITLE

Exploring Core Modularity in Node.js: OS, Path, and File System (fs) Modules

2. AIM

To explore the concept of Node.js modularity and implement practical applications using the built-in os, path, and fs modules for extracting system information, managing file paths across platforms, and performing file I/O operations.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the architecture of Node.js modules and the CommonJS module system.
 - To utilize the built-in 'os' module to retrieve operating system platform, CPU architecture, memory, and uptime.
 - To utilize the built-in 'path' module for cross-platform path normalization, extraction, parsing, and joining.
 - To implement synchronous file creation, appending, reading, and deletion using the built-in 'fs' module.
 - To combine multiple core modules into a single consolidated Node.js script.
-

4. THEORY

Node.js Modules

In Node.js, a module is an encapsulated, reusable block of code whose variables, functions, and classes are private to the file unless explicitly exported. Modularity facilitates code maintainability, separation of concerns, and reuse. Node.js supports three types of modules:

1. Core (Built-in) Modules: Pre-bundled with Node.js binary (e.g., os, path, fs, http). They require no installation and are imported using `require("module_name")`.

2. Local Modules: Custom files created within the application and exported using `module.exports` and imported using relative paths (e.g., `require('./myModule')`).
3. Third-Party Modules: Packages published by the community on NPM and installed via `npm install` (e.g., `express`, `chalk`).

Purpose & Examples of Core Modules

1. OS Module ('os') — Provides operating system-related utility methods and properties. It allows applications to query host hardware, platform name, memory statistics, and CPU specs. Example: `os.platform()` returns the OS platform ('win32', 'linux'); `os.freemem()` returns available system RAM in bytes; `os.cpus()` lists CPU core configurations.

2. Path Module ('path') — Provides utilities for handling and transforming file and directory paths. It resolves platform-specific delimiter discrepancies (Windows '\\' vs POSIX '/'). Example: `path.join(__dirname, 'projects', 'file.txt')` safely joins path segments; `path.extname(path)` extracts the file extension ('.txt'); `path.parse(path)` decomposes a path string into root, dir, base, ext, and name.

3. File System Module ('fs') — Enables interaction with the host file system for reading, writing, updating, and deleting files and directories. Provides synchronous and asynchronous methods. Example: `fs.writeFileSync()` writes text to disk; `fs.readFileSync()` retrieves file contents; `fs.appendFileSync()` adds text to the end of a file; `fs.unlinkSync()` deletes a file.

5. ALGORITHM / PROCEDURE

- Create a JavaScript file named `nodeModulesDemo.js`.
 - Import the built-in modules using `require('os')`, `require('path')`, and `require('fs')`.
 - Section 1 (OS Module): Call `os.platform()`, `os.release()`, `os.type()`, `os.arch()`, `os.cpus().length`, `os.totalmem()`, `os.freemem()`, `os.homedir()`, `os.uptime()`, and `os.userInfo().username` and print results.
 - Section 2 (Path Module): Construct an absolute path using `path.join(__dirname, 'projects', 'document.notes.txt')`. Extract `dirname`, `basename`, and `extname`. Parse the path using `path.parse()` and reconstruct it with `path.format()`.
 - Section 3 (FS Module): Construct a sample file path `sample.txt` using `path.join(__dirname, 'sample.txt')`.
 - Write initial content to `sample.txt` using `fs.writeFileSync()`.
 - Append an additional line to `sample.txt` using `fs.appendFileSync()`.
 - Read and log the contents of `sample.txt` using `fs.readFileSync(..., 'utf8')`.
 - Clean up the temporary file using `fs.unlinkSync()`.
 - Run `node nodeModulesDemo.js` in terminal and record the output.
-

6. PROGRAM

nodeModulesDemo.js

```
// Import built-in Node.js modules
const os = require("os");
const path = require("path");
const fs = require("fs");

// 1. OS MODULE

console.log("=== OS MODULE DETAILS ===");

// Operating System Identification
console.log("Platform (OS Name):", os.platform());
console.log("OS Release:", os.release());
console.log("OS Type:", os.type());

// CPU Information
console.log("Architecture:", os.arch());
console.log("Number of CPU Cores:", os.cpus().length);

// Memory Information
console.log("Total Memory :", os.totalmem());
console.log("Free Memory :", os.freemem());

// Environment & System Context
console.log("Home Directory:", os.homedir());
console.log("System Uptime:", os.uptime());
console.log("Current User:", os.userInfo().username);

// 2. PATH MODULE

console.log("\n=== PATH MODULE DETAILS ===");

// Construct path safely across platforms
let builtPath = path.join(__dirname, "projects", "document.notes.txt");
console.log("Joined Absolute Path:", builtPath);

// Extract individual components
console.log("Directory Name:", path.dirname(builtPath));
console.log("File Name:", path.basename(builtPath));
console.log("File Extension:", path.extname(builtPath));
```

```
// Parse path into an object and format back to string
let parsedPath = path.parse(builtPath);
console.log("Parsed Path Object:", parsedPath);
console.log("Reconstructed Path:", path.format(parsedPath));


// 3. FS MODULE

console.log("\n=== FS MODULE DETAILS ===");

// Write content to a file
fs.writeFileSync(sampleFile, "Hello, Node.js!\nThis is a detailed FS module demonstration.");
console.log("File created and initial content written.");

// Append additional content
fs.appendFileSync(sampleFile, "\nAppended Line: Learning file sync operations.");
console.log("Content appended to file.");

// Read file contents
let fileData = fs.readFileSync(sampleFile, "utf8");
console.log("\n--- File Contents ---");
console.log(fileData);

// Clean up created file and directory
fs.unlinkSync(sampleFile);
```

7. CODE EXPLANATION

Code / Component	Explanation
os.platform() / os.arch()	Returns OS platform (e.g. 'win32') and CPU instruction architecture (e.g. 'x64').
os.totalmem() / os.freemem()	Returns total system memory and unallocated RAM in bytes.
path.join(...)	Concatenates all given path segments using the platform-specific separator.
path.dirname() / path.basename()	Extracts directory path and trailing file name segment.

<code>path.extname() / path.parse()</code>	Extracts extension ('.txt') and parses full path into structured properties.
<code>fs.writeFileSync()</code>	Synchronously creates or overwrites a target file with specified text.
<code>fs.appendFileSync()</code>	Appends new data to the end of an existing file.
<code>fs.readFileSync(..., 'utf8')</code>	Synchronously reads file contents and decodes as string.
<code>fs.unlinkSync()</code>	Synchronously removes/deletes the specified file from the filesystem.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Execute OS methods	Prints host OS platform, architecture, RAM, and username	System specs logged accurately	Pass
TC-02	Execute <code>path.join()</code> and <code>path.parse()</code> in <code>path_module.js</code>	Terminal displays normalized path and parsed object containing root, dir, base, ext, and name	Path constructed and parsed successfully	Pass
TC-03	Write, append, and read using fs	Creates <code>sample.txt</code> , appends line, and returns full text	File content matches input stream	Pass
TC-04	Unlink sample file	<code>sample.txt</code> deleted	File removed without errors	Pass

9. OUTPUT

Output 1: Execute OS methods

Detailed terminal console logs showing outputs from OS modules

```
Problems  Debug Console  Output  Terminal  Ports
● PS C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-6> node modules.js
=== OS MODULE DETAILS ===
Platform (OS Name): win32
OS Release: 10.0.26200
OS Type: Windows_NT
Architecture: x64
Number of CPU Cores: 12
Total Memory : 16471101440
Free Memory : 4061691904
Home Directory: C:\Users\Purna
System Uptime: 129825.843
Current User: Purna
```

Output 2: Execute path.join() and path.parse() in path_module.js

Terminal displays normalized path and parsed object containing root, dir, base, ext, and name

```
=== PATH MODULE DETAILS ===
Joined Absolute Path: C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-6\projects\document.notes.txt
Directory Name: C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-6\projects
File Name: document.notes.txt
File Extension: .txt
Parsed Path Object: {
  root: 'C:\\',
  dir: 'C:\\Users\\Purna\\Documents\\Projects\\FullStack\\Observation\\Task_6\\PBL-6\\projects',
  base: 'document.notes.txt',
  ext: '.txt',
  name: 'document.notes'
}
Reconstructed Path: C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-6\projects\document.notes.txt
```

Output 3: Write, append, and read using fs

Creates sample.txt, appends line, and returns full text

```
=== FS MODULE DETAILS ===
File created and initial content written.
Content appended to file.

--- File Contents ---
Hello, Node.js!
This is a detailed FS module demonstration.
Appended Line: Learning file sync operations.
```

Output 4: Unlink sample file

File removed without errors.

10. RESULT

Thus, the concept of modularity in Node.js was comprehensively explored, and a Node.js program utilizing the core `os`, `path`, and `fs` modules was successfully implemented and verified. The program demonstrates host inspection, reliable cross-platform path resolution, and synchronous file I/O operations. All test cases passed with expected outputs.